BreachMirror AI: Security Handbook for Internees & New Employees

Zero-Blame Autonomous Cyber Defense & Threat Prevention Guide

---

Executive Summary & The Zero-Blame Mandate

Welcome to the team! As an internee or new employee, your daily work directly impacts the security of our platform, our customers, and our infrastructure.

Our Core Philosophy: Zero-Blame Culture

1. **Speed matters more than fault:** An accidental secret commit reported in 5 minutes can be rotated in 60 seconds with zero damage. A secret kept hidden for 3 days leads to massive data theft.
2. **Honest mistakes are learning opportunities:** You will never be penalized, reprimanded, or shamed for reporting an honest security mistake.
3. **Emergency Channel:** Whenever in doubt, post in `#security-emergency` or call the SOC hotline immediately.

---

1. Generative AI & LLM Data Ingestion (AI & Machine Learning)

Risk Level: CRITICAL *Audience Note:* Most frequent vulnerability for new joiners eager to write code or draft documents quickly.

How the Breach Occurs:

Pasting proprietary source code, database credentials, internal system prompts, or customer PII into public AI chat interfaces (e.g. public ChatGPT, Claude, Copilot web).

Common Scenarios:

- Pasting a complex SQL query containing production database hostnames and passwords to ask AI to optimize it.
- Pasting internal customer support chats containing customer emails, credit card fragments, or health data to summarize.
- Uploading unreleased patent documents, API schemas, or financial projections for grammar correction.

Underlying Mechanics: Consumer AI tiers store prompts to retrain public models. If training data is subsequently extracted or logs are compromised, proprietary code is permanently exposed.

Downstream Blast Radius:

- **Business Impact:** IP forfeiture, loss of competitive edge, exposed system architecture enabling targeted zero-day exploits.
- **Data at Risk:** Source code, API secrets, customer PII, internal architecture diagrams.
- **Compliance & Legal:** Severe violations of GDPR, HIPAA, SOC 2 Type II, and client NDAs with multimillion-dollar fines.

How to Prevent It:

Golden Rules:

1. Never paste real credentials, tokens, or proprietary logic into unsanctioned consumer AI models.
2. Use only the company-sanctioned Enterprise AI Gateway with zero-retention contractual agreements.
3. Sanitize and redact: replace real secrets with <REDACTED_API_KEY> and mock table names before querying AI.

Approved Tools & Alternatives:

- `Use the enterprise sandbox gateway: https://ai-sandbox.internal.corp (zero logging)`
- `Local offline models via Ollama running on secure corporate hardware`

DOs:

- [x] **DO:** Ask AI for algorithmic syntax using generic abstract examples (e.g. "How do I invert a binary tree in TypeScript?").
- [x] **DO:** Use synthetic dummy data when asking for regexes or parsing logic.
- [x] **DO:** Verify with SecOps before installing third-party AI browser extensions or IDE plugins.

DONTs:

- [ ] **DON'T:** DO NOT paste code snippets with private company package names, internal URLs, or tokens.
- [ ] **DON'T:** DO NOT upload internal PDFs, confidential roadmap presentations, or customer spreadsheets.
- [ ] **DON'T:** DO NOT disable IDE security linters that warn of credential ingestion.

---

2. Git Commits & Hardcoded Secrets (Source Control & CI/CD)

Risk Level: CRITICAL *Audience Note:* Common for internees pushing their first PRs or working on local configuration files.

How the Breach Occurs:

Committing API keys, AWS/GCP service account JSON files, SSH private keys, or `.env` files into Git repositories (public repos, personal forks, or shared branches).

Common Scenarios:

- Temporarily hardcoding `const STRIPE_KEY = "sk_live_..."` while debugging and committing with `git commit -am "fixed payment"`.
- Pushing a `.env` file that was forgotten because `.gitignore` was not initialized first.
- Pushing code to a personal public GitHub repository to "save homework" or share with a friend.

Underlying Mechanics: Automated adversarial web scrapers monitor the public GitHub event stream 24/7. Pushed credentials are stolen and exploited within 2 to 15 seconds.

Downstream Blast Radius:

- **Business Impact:** Unauthorized cloud infrastructure access, cryptomining spin-ups incurring tens of thousands of dollars in hours, complete database exfiltration.
- **Data at Risk:** Cloud master keys, Stripe/payment credentials, database root access.
- **Compliance & Legal:** Immediate breach disclosure mandates (72h under GDPR), loss of PCI-DSS certification.

How to Prevent It:

Golden Rules:

1. Never write raw secrets in source code files. Secrets must always be injected at runtime via environment variables.
2. Always ensure `.env` and `*.pem` are listed in `.gitignore` before creating your first local commit.
3. Install and configure pre-commit hooks (`git-secrets`, `gitleaks`) on your development machine.

Approved Tools & Alternatives:

- `HashiCorp Vault or AWS Secrets Manager for secret retrieval`
- ``direnv `or company Doppler workspace for local secret injection`

DOs:

- [x] **DO:** Check `git status` and `git diff --staged` before every single `git commit`.
- [x] **DO:** Use `.env.example` with blank values to document required environment variables.
- [x] **DO:** If you accidentally commit a secret, notify SecOps immediately to rotate the credential. (Git history rewrite alone is NOT sufficient!).

DONTs:

- [ ] **DON'T:** DO NOT assume a repo being "private" makes it safe to hardcode secrets.
- [ ] **DON'T:** DO NOT push company source code to personal GitHub, GitLab, or Gist accounts.
- [ ] **DON'T:** DO NOT share `.env` files via Slack, email, or chat messages.

---

3. Phishing, AiTM & Reverse-Proxy Portals (Authentication & Identity)

Risk Level: CRITICAL *Audience Note:* Internees and new employees are prime targets for fake onboarding, payroll, or IT verification emails.

How the Breach Occurs:

Adversary-in-the-Middle (AiTM) phishing links clone the corporate Okta or Google SSO page in real time to capture credentials AND session cookies, bypassing legacy SMS or app push MFA.

Common Scenarios:

- Receiving an email: "Urgent: Complete your new joiner benefits enrollment by 5 PM" linking to `okta-company-benefits-login.com`.
- Receiving a Google Calendar invite or DocuSign notification from an external domain with an attached sign-in link.
- Approving an unsolicited Okta Push notification on your phone after someone attempts password spray (MFA Fatigue).

Underlying Mechanics: Reverse proxy frameworks (Evilginx) proxy traffic between the victim and real login service. When you complete MFA, the attacker captures the authenticated session token.

Downstream Blast Radius:

- **Business Impact:** Full lateral movement across Google Drive, Slack, GitHub, and internal VPN without triggering standard password alerts.
- **Data at Risk:** Entire corporate email archive, internal chats, confidential employee records.
- **Compliance & Legal:** Identity compromise across all federated corporate services.

How to Prevent It:

Golden Rules:

1. Inspect the browser address bar: Company SSO is ONLY hosted on `https://company.okta.com` or `https://accounts.google.com`.
2. Use FIDO2 WebAuthn hardware keys (YubiKeys) or device biometric passkeys—they are cryptographically bound to the domain and mathematically immune to AiTM proxies.
3. Never approve an MFA push notification that you did not initiate yourself in that exact moment.

Approved Tools & Alternatives:

- `Hardware Security Keys (YubiKey 5 Series)`
- `Built-in Touch ID / Windows Hello WebAuthn credentials`

DOs:

- ☑ **DO:** Bookmark the authentic SSO portal and access it only via your bookmarks.
- ☑ **DO:** Report suspicious emails immediately using the "Report Phish" button in your email client.

- ☑ **DO:** If you clicked a suspect link, change your password immediately and notify `#security-emergency`.

DONTs:

- ☐ **DON'T:** DO NOT approve multiple consecutive MFA push requests (MFA Fatigue attack).
- ☐ **DON'T:** DO NOT log into corporate accounts from personal devices or untrusted public computers.
- ☐ **DON'T:** DO NOT trust emails solely because they feature the corporate logo and executive names.

---

4. Shadow IT & Cloud File Sharing (Cloud Security & DLP)

Risk Level: HIGH *Audience Note:* Frequent slip-up when trying to share files with contractors, teammates, or external mentors.

How the Breach Occurs:

Changing Google Drive or cloud storage links to "Anyone with the link can view", or deploying S3/Cloud Storage buckets with public read ACLs in infrastructure code.

Common Scenarios:

- Sharing an internal roadmap or architecture design with an outside mentor by toggling "Anyone on the Internet with this link can view".
- Setting `acl = "public-read"` in Terraform to quickly serve an image asset or test build artifact.
- Using unauthorized personal cloud storage (Dropbox, iCloud, Mega) to transfer work files home.

Underlying Mechanics: Public links are easily crawled by search engines, leak via HTTP referer headers, or are discovered through cloud enumeration tools.

Downstream Blast Radius:

- **Business Impact:** Silent exposure of confidential business plans, unreleased product source code, or internal customer databases.
- **Data at Risk:** Company financial records, client NDAs, design blueprints.
- **Compliance & Legal:** Breach of contract with enterprise customers and data privacy regulations.

How to Prevent It:

Golden Rules:

1. Keep cloud sharing strictly scoped to "@company.com" or explicitly named collaborators.
2. Never set public read or write permissions on cloud storage buckets without SecOps sign-off.
3. Use only IT-sanctioned file repositories; personal drives and shadow cloud tools are strictly prohibited.

Approved Tools & Alternatives:

- `Enterprise Google Workspace / Box with strict organizational domain boundaries`
- `Internal artifact repositories with authenticated presigned URLs (expiring in 15 minutes)`

DOs:

- ☑ **DO:** Regularly review your shared Google Drive documents and revoke access when projects end.
- ☑ **DO:** Always use role-based least privilege IAM policies for cloud storage.
- ☑ **DO:** Verify file sharing settings show "Restricted" or "Company domain only".

DONTs:

- ☐ **DON'T:** DO NOT email work spreadsheets or code tarballs to your personal Gmail or iCloud account.
- ☐ **DON'T:** DO NOT create "Anyone with link" access for internal documents.
- ☐ **DON'T:** DO NOT host corporate databases or backups on public S3/GCS buckets.

---

5. Removable Hardware, USBs & Public Wi-Fi (Endpoint & Physical Security)

Risk Level: HIGH *Audience Note:* Critical for remote workers, coffee shop coding, and in-office physical safety.

How the Breach Occurs:

Plugging in an untrusted USB flash drive, working on confidential code over unsecured public Wi-Fi without VPN, or leaving laptops unlocked in public areas.

Common Scenarios:

- Finding a branded USB drive in the office lobby or coffee shop and inserting it to "see who owns it".
- Connecting to free "Airport_Free_WiFi" without turning on the corporate VPN, allowing DNS spoofing and packet sniffing.
- Leaving a corporate laptop unlocked while getting coffee at a café or coworking space.

Underlying Mechanics: Malicious USB devices (Rubber Ducky, Bash Bunny) emulate HID keyboards and inject hundreds of keystrokes per second, spawning reverse shells or exfiltrating browser data in seconds.

Downstream Blast Radius:

- **Business Impact:** Local machine compromise, silent installation of persistent keyloggers, lateral spread to adjacent corporate subnet.
- **Data at Risk:** Stored browser passwords, SSH keys, local cached source repositories.

- **Compliance & Legal:** Direct failure of physical security audits (ISO 27001).

How to Prevent It:

Golden Rules:

1. Never connect an unknown or personal USB storage drive to a corporate machine. (Corporate devices enforce kernel-level USB block).
2. Always lock your screen (`Win + L` or `Cmd + Ctrl + Q`) whenever you step away from your workstation.
3. Always enable the Corporate Zero-Trust VPN / ZTNA client when connecting to any Wi-Fi outside the corporate office.

Approved Tools & Alternatives:

- `Company Cloud Storage for file transfers`
- `Encrypted corporate hardware provisioned by IT Support`

DOs:

- [x] **DO:** Hand any found USB drives directly to the physical security desk.
- [x] **DO:** Use privacy screen filters when working in airports, trains, or public cafés.
- [x] **DO:** Keep your laptop in your possession or in a locked room at all times.

DONTs:

- [ ] **DON'T:** DO NOT insert USB drives given as vendor swag at conferences until scanned by IT.
- [ ] **DON'T:** DO NOT disable disk encryption (FileVault / BitLocker) or firewall settings.
- [ ] **DON'T:** DO NOT leave laptops unattended in parked vehicles.

---

6. Dependency Poisoning & Typosquatting (Supply Chain Security)

Risk Level: HIGH *Audience Note:* Crucial for developers installing third-party packages and libraries.

How the Breach Occurs:

Installing a typosquatted npm/PyPI package (e.g. `cross-env` vs `crossenv`, `colorama` vs `colourama`) or copy-pasting install commands from unverified blogs.

Common Scenarios:

- Typing `npm install react-rotuer` by mistake when installing react-router.
- Copying a `curl ... | bash` command from a random blog tutorial to install a development utility.
- Importing a GitHub repo directly without pinning a specific commit hash or tag.

Underlying Mechanics: Malicious package authors register names with common misspellings. When installed, `preinstall` or `postinstall` lifecycle scripts execute arbitrary code, extracting `.env` secrets.

Downstream Blast Radius:

- **Business Impact:** Supply chain injection: malicious packages can be bundled into production releases and distributed to millions of end-users.
- **Data at Risk:** Developer machine environment variables, SSH keys, CI/CD publish tokens.
- **Compliance & Legal:** Severe supply chain liability, software provenance compliance failure.

How to Prevent It:

Golden Rules:

1. Double-check package spellings against the official documentation before running `npm install` or `pip install`.
2. Always verify package popularity: check weekly downloads, GitHub stars, and verified publisher badges.
3. Route all package installations through the company internal artifact registry proxy which scans for known CVEs.

Approved Tools & Alternatives:

- `Internal Artifact Registry (JFrog Artifactory / GitHub Packages)`
- `Lockfiles (` package-lock.json `,` pnpm-lock.yaml `) with SHA integrity hashes`

DOs:

- [x] **DO:** Commit lockfiles to ensure deterministic and tamper-evident builds.
- [x] **DO:** Run `npm audit` or `snyk test` before introducing any new open-source dependency.
- [x] **DO:** Inspect the `package.json` scripts of unknown third-party libraries.

DONTs:

- [ ] **DON'T:** DO NOT pipe untrusted curl commands to bash (`curl http://... | sh`).
- [ ] **DON'T:** DO NOT install packages with fewer than 1,000 downloads without Tech Lead review.
- [ ] **DON'T:** DO NOT bypass security linter warnings regarding vulnerable dependencies.

---

7. Social Engineering & Executive Impersonation (Human Factors & Operations)

Risk Level: HIGH *Audience Note:* Internees are targeted because attackers know they are hesitant to question senior leadership.

How the Breach Occurs:

Attackers create fake Slack, Teams, WhatsApp, or LinkedIn accounts pretending to be the CEO, VP of Eng, or HR Director, demanding urgent action outside standard workflows.

Common Scenarios:

- Receiving a message: "Hey, it's the CTO. I'm locked out and need you to push this commit directly to main bypassing branch protection."
- Receiving a WhatsApp text: "I am in an investor meeting and need you to purchase 5 prepaid cards or approve this wire transfer ASAP."
- An alleged coworker asks you to share your VPN credentials or send an OTP code they accidentally sent to your phone.

Underlying Mechanics: Attackers exploit psychological compliance, artificial urgency, and the fear of saying "no" to a high-ranking executive.

Downstream Blast Radius:

- **Business Impact:** Bypassing code review and CI/CD controls, wire fraud, compromised internal communications.
- **Data at Risk:** Bypassed pull request controls, financial assets, internal access tokens.
- **Compliance & Legal:** SOX compliance failures, operational control violations.

How to Prevent It:

Golden Rules:

1. Remember: Executives will NEVER ask an internee or new employee to bypass security controls, push directly to main, or purchase gift cards.
2. Always verify unusual or out-of-process requests using a separate communication channel (e.g. video call, in-person, official corporate phone).
3. Never share OTPs, MFA codes, or credentials with ANYONE, even if they claim to be IT Support or Executive Leadership.

Approved Tools & Alternatives:

- `Standard GitHub Pull Request workflows with required 2-peer approvals`
- `Official IT Helpdesk ticketing system for access requests`

DOs:

- ☑ **DO:** Check the sender's full email address or Slack handle (look for external guest tags).
- ☑ **DO:** Pause and discuss the request with your immediate manager or mentor.
- ☑ **DO:** Take pride in verifying: security-conscious verification is respected and praised, never punished.

DONTs:

- ☐ **DON'T:** DO NOT let artificial urgency force you into skipping security procedures.
- ☐ **DON'T:** DO NOT communicate with executives via non-company messaging apps (WhatsApp/Telegram).
- ☐ **DON'T:** DO NOT give your MFA codes to anyone claiming to be "IT verifying your account".

Security Emergency Contacts

- **Slack / Teams:** `#security-emergency` (Monitored 24/7)
- **SOC Incident Hotline:** `ext. 4444`
- **Incident Email:** `security-incident@internal.corp`